

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

(Grupo 1)

Trabalho de Grupo de Sistemas Operativos

Elaborado por:

[Gonçalo Marques - 52232]

[Rodrigo Lourenço - 51687]

[Uberaldo Kill - 51784]

Professor Orientador:

Professor Doutor Paul Crocker

Conteúdo

Conteúdo	0
1 Resumo	1
2 Introdução	3
2.1 Constituição do Grupo	3
2.2 Organização do Documento	3
3 Engenharia de Software	5
3.1 Estado da Arte	5
3.2 Arquitectura e fluxo detalhado	5
3.3 Estruturas de dados principais	6
4 Implementação	7
4.1 Trechos de implementação relevantes	7
4.1.1 Escalonador — comparadores	7
4.1.2 Execução de instrução e fork/exec	7
4.1.3 Gestão de memória — carga, compactação e partições	8
4.2 Bloqueio e desbloqueio de processos	8
4.3 Exemplos de experiências e interpretação de resultados	9
4.3.1 Exemplo 1 — SJF preemptivo vs FCFS	9
4.3.2 Como extrair métricas	9
5 Conclusões e Trabalho Futuro	11
5.1 Conclusões	11
5.2 Trabalho Futuro	11
.1 Apêndice A — Lista de ficheiros relevantes	12
.2 Apêndice B — Exemplo de ‘data/control.txt’	13
Bibliografia	15

Capítulo

1

Resumo

Este relatório detalhado descreve a implementação e avaliação de um simulador de escalonamento de processos desenvolvido a partir do código-fonte presente no repositório. O documento inclui: arquitectura, explicação das estruturas de dados, descrição de cada módulo, trechos de código relevantes, instruções de compilação/execução, exemplos de utilização, e análise teórica dos algoritmos de escalonamento (com métricas e exemplos concretos).

Divisão de Trabalho:

- Rodrigo Lourenço - Responsável pelo escalonador, filas de prioridade e validação dos algoritmos e Escrita do Relatório;
- Gonçalo Marques — Gestor e Memória;
- Uberaldo Kill — Loader/Memory e testes;
- Todos — análise de resultados, escrita do relatório e revisão final.

Link GitHub:<https://github.com/Rodralex10/SOGrupo1>

Capítulo

2

Introdução

Este projeto implementa um simulador didático de um sistema operativo em escalonamento de processos e gestão de memória. O objetivo é permitir a experimentação e a demonstração prática dos efeitos de diferentes políticas de escalonamento (FCFS, Priority, SJF, RM, EDF), bem como a investigação de estratégias de alocação de memória e do comportamento de operações *'fork'* e *'exec'* simuladas pelas instruções *'C'* e *'L'*.

O relatório segue um formato académico: descrição do problema, estado da arte, arquitectura, implementação detalhada (com trechos de código), exemplos de execução, análise teórica, resultados e conclusões.

2.1 Constituição do Grupo

O grupo é composto pelos seguintes elementos:

- Gonçalo Marques - 52232
- Rodrigo Lourenço - 51687
- Uberaldo Kill - 51784

2.2 Organização do Documento

Este relatório está organizado em capítulos principais (cada capítulo corresponde a uma unidade temática):

- **Capítulo 1 (Introdução)** - Descrição geral da proposta, constituição do grupo e organização do documento.

- **Capítulo 2 (Engenharia de Software)** - Estado da arte, ferramentas e tecnologias utilizadas, requisitos e arquitetura do sistema.
- **Capítulo 3 (Implementação)** - Detalhes da implementação, trechos de código relevantes, gestão de memória, escalonamento, fork/exec e manuais de instalação.
- **Capítulo 4 (Reflexão Crítica e Problemas Encontrados)** - Análise dos objetivos alcançados, divisão de trabalho e problemas encontrados durante o desenvolvimento.
- **Capítulo 5 (Conclusões e Trabalho Futuro)** - Conclusões e propostas de trabalho futuro.

Capítulo

3

Engenharia de Software

3.1 Estado da Arte

Resumo das políticas de escalonamento implementadas e suas características:

- FCFS: simplicidade, justa por ordem de chegada, má para processos curtos.
- Priority: sensível à escolha de prioridades; pode causar starvation.
- SJF: otimiza turnaround médio, mas requer estimativas do comprimento; versão preemptiva é conhecida como SRTF.
- RM: algoritmo para tarefas periódicas com prioridades estáticas baseadas no período.
- EDF: algoritmo dinâmico que prioriza prazos; ótimo para tarefas com deadlines.

3.2 Arquitetura e fluxo detalhado

O simulador está organizado em módulos (ficheiros em 'src/')

- 'manager.c' — inicialização, leitura do plano, ciclo principal, comandos e coordenação entre módulos.
- 'scheduler.c' — funções de comparação, enfileiramento, seleção de próximo processo (curto e longo prazo).

- ‘process.c’ — alocação de PCB, execução de uma instrução (*process_exec_one*), implementação de C (fork) e ‘L’ (load/exec).
- ‘memory.c’ / ‘loader.c’ — gestão do array de instruções, ‘prog_slots’, compactação e carregamento de ficheiros ‘.prg’.
- ‘queues.c’, ‘report.c’ — utilitários de filas e geração de relatórios.

Fluxo de alta-nível:

1. ‘manager_init’ configura estruturas, carrega ficheiros de configuração e o plano.
2. No ciclo principal (‘manager_run’), o simulador lê comandos de ‘control.txt’ ou ‘stdin’.
3. ‘E’ (executar) invoca ‘manager_exec_quantum’ que executa até ‘time_quantum’ ticks; cada tick chama ‘process_exec_one’ para a instrução atual.
4. Eventos (bloqueio, término, fork, load) são tratados e reflectidos nas filas (ready, blocked, wait_mem, terminated).

3.3 Estruturas de dados principais

Apresentamos os tipos centrais definidos em include/types.h.

```
typedef struct {
    int pid; int ppid;
    char program_name[MAX_NAME];
    int memory_start; int memory_len; int memory_size;
    int priority; int pc; int var;
    int arrival_time; int start_time; int finish_time;
    int cpu_time_used; int remaining;
    int period; int deadline; int next_release;
    ProcState state; int active;
} PCB;
```

Listing 3.1: PCB (resumo) — trechos de include/types.h

Também existe o tipo ‘Instruction’ representando uma célula de ‘memory[]’:

```
typedef struct {
    InsOp op; int n; char nome[MAX_NAME];
} Instruction;
```

Listing 3.2: Instruction (resumo)

Capítulo

4

Implementação

4.1 Trechos de implementação relevantes

Nesta secção extraímos e comentamos os pontos mais importantes do código.

4.1.1 Escalonador — comparadores

O arquivo 'src/scheduler.c' contém funções de comparação usadas pelo heap de prioridade. Exemplo (simplificado):

```
int cmp_sjf(int a, int b, const PCB *tbl) {
    int diff = tbl[a].remaining - tbl[b].remaining;
    if (diff != 0) return diff;
    return tbl[a].arrival_time - tbl[b].arrival_time;
}
```

Listing 4.1: *cmp_{sjf}—menorremaining = maisprioritário*

4.1.2 Execução de instrução e fork/exec

O núcleo do intérprete está em 'src/process.c'. Os comportamentos 'C' e 'L' seguem estas regras:

- 'C n': o pai salta para 'pc = pc + 1 + n'; o filho recebe 'pc = parent_pc + 1' e é criado com novo PID. O slot de programa é partilhado (ref_count++).
- 'L f': o processo liberta o slot anterior ('memory_release') e tenta carregar 'f' com 'memory_load_program'. Se falhar, termina.

Trecho (simplificado):

```
case INS_C: {
    int child_pc = p->pc + 1;
    p->pc = p->pc + 1 + ins.n;
    int ci = process_fork(idx, pcb_table, pcb_count, new_pid, sim_time);
    pcb_table[ci].pc = child_pc;
    return 3; // sinaliza cria o de filho
}

case INS_L: {
    memory_release(p->program_name, pcb_table, *pcb_count);
    int start = memory_load_program(ins.nome, pcb_table, *pcb_count);
    if (start < 0) { p->state = PROC_TERMINATED; return 2; }
    p->memory_start = start; p->pc = 0; strncpy(p->program_name, ins.nome,
        ...);
    return 4; // sinaliza load
}
```

Listing 4.2: Manuseamento de 'C' e 'L' em *process_execone*

4.1.3 Gestão de memória — carga, compactação e partições

O 'memory.c' implementa:

- 'memory_load_program': procura slot existente, conta instruções, encontra região livre (ou faz compactação) e carrega com 'loader_load'.
- 'memory_compact': desloca programas para o início do array e actualiza 'memory_start' nos PCBs.
- 'partition_alloc' / 'partition_release': suporte a partições fixas (first-fit).

Trecho (compactação — simplificado):

4.2 Bloqueio e desbloqueio de processos

Bloquear: instrução 'B' define 'p->state = PROC_BLOCKED' e enfileira em 'blocked_queue'. Desbloquear é realizado pelo comando 'D' que chama 'sched_long', que escolhe aleatoriamente um processo bloqueado e passa-o para 'READY' (adicionando à fila de prontos apropriada). O comando 'D' também tenta recarregar processos que aguardavam memória ('WAIT_MEM').

4.3 Exemplos de experiências e interpretação de resultados

Aqui indicamos exemplos simples e como obter métricas do simulador.

4.3.1 Exemplo 1 — SJF preemptivo vs FCFS

Plano (exemplo reduzido) — edite 'data/plan.txt' como:

```
P1.prg 0 5  
P2.prg 1 5
```

Onde 'P1.prg' e 'P2.prg' têm comprimentos 5 e 2 instruções, respectivamente.

Com o simulador em modo SJF e preemptivo, espera-se que P2 preempça P1 na chegada e reduza o turnaround médio. Recolha as variáveis 'start_time' e 'finish_time' do 'report' para calcular 'turnaround = finish_time - arrival_time'.

4.3.2 Como extrair métricas

O módulo 'report.c' imprime estatísticas globais; alternativamente, pode-se imprimir para ficheiro as linhas de log e processá-las para calcular:

- Turnaround: 'finish_time - arrival_time'
- Waiting: 'start_time - arrival_time' (ou soma de períodos em READY)
- CPU time: 'cpu_time_used'

Conclusões e Trabalho Futuro

5.1 Conclusões

O simulador extraído do projecto implementa os requisitos pedidos pelo docente: vários algoritmos de escalonamento, fork/exec simulados, gestão de memória com compactação e partições, e mecanismos de bloqueio/desbloqueio. O ficheiro `REPORT.tex` foi expandido com trechos do código e instruções para reprodução das experiências.

5.2 Trabalho Futuro

Sugestões de melhorias e extensões:

- Adicionar medição automática e geração de gráficos (via scripts Python).
- Implementar escalonadores adicionais e políticas de envelhecimento para evitar starvation.
- Suporte a simulação multi-core e sincronização entre núcleos.

.1 Apêndice A — Lista de ficheiros relevantes

src/manager.c (src/manager.c)

src/scheduler.c (src/scheduler.c)

src/process.c (src/process.c)

src/memory.c (src/memory.c)

src/loader.c (src/loader.c)

include/types.h (include/types.h)

Makefile (Makefile)

.2 Apêndice B — Exemplo de ‘data/control.txt’

```
# Sequência de comandos: E=executar, R=relatório, T=terminar
E
E
R
D
E
T
```


Bibliografia

<https://developer.android.com/docs>
<https://developer.android.com/studio>
<https://developers.google.com/maps>
<https://developers.google.com/maps/documentation/android-sdk>
<https://developers.google.com/location-context/fused-location-provider>
<https://developer.tomtom.com>
<https://developer.tomtom.com/traffic-api/documentation>
<https://developer.tomtom.com/routing-api/documentation>
<https://supabase.com/docs>
<https://supabase.com/docs/guides/auth>
<https://supabase.com/docs/guides/database>
<https://supabase.com/docs/guides/auth/row-level-security>
<https://supabase.com/docs/guides/api>